

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I **H.Mohammed Nawaz (192325060)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **H.Mohammed Nawaz (192325060)**

Assessment Tasks Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective	Reasonable	Basic corrections	Ineffective

1. Hadoop Job Fails Due to Uneven Input Splits

The Hadoop job is likely failing because the input data is unevenly distributed among input splits. Some Mapper tasks receive very large amounts of data while others receive smaller amounts. The large splits take longer to finish, causing data skew and poor resource utilization. As a result, some Reducers remain idle while waiting for Mapper output.

The main design error may be an inappropriate HDFS block size, uneven file sizes, or a highly skewed input dataset. Very large files or poorly distributed records can create unbalanced workloads.

To correct this problem, the input files should be divided into reasonably sized HDFS blocks. The number of Mapper tasks should be adjusted according to the available cluster resources. A suitable Combiner can reduce intermediate data before it reaches the Reducers.

The partitioning logic should also be checked to ensure that data is distributed evenly among Reducers. If one key contains a very large number of records, a better partitioning strategy should be used.

Conclusion: The problem is mainly caused by data or task imbalance. Proper block sizing, balanced input files, suitable partitioning, and optimized Mapper/Reducer configuration can improve performance and prevent repeated failures.

2. HDFS Data Unavailability After a Single Node Failure

Frequent data unavailability after one DataNode fails indicates a possible replication or NameNode design problem. HDFS should normally maintain multiple copies of important data blocks so that failure of one DataNode does not make the data unavailable.

One possible error is that the replication factor is too low, such as being set to 1. If the only copy of a block exists on the failed node, the data cannot be accessed. Another possibility is that replicas are placed on the same physical node or rack, reducing fault tolerance.

The HDFS configuration should use an appropriate replication factor, normally **3 for important data**, while staying within storage constraints. Replicas should be distributed across different DataNodes and racks using **rack awareness**.

The NameNode should also have **High Availability**, using Active and Standby NameNodes. Automatic failover should be configured so that a NameNode failure does not interrupt access to HDFS metadata.

Regular health monitoring and replication checks should be performed to identify under-replicated blocks.

Conclusion: Proper replication, rack-aware block placement, NameNode High Availability, and continuous monitoring can prevent data unavailability after a single-node failure.

3. Duplicate Records from ETL and Apache NiFi

Duplicate records in the analytics store can occur because the ETL pipeline or Apache NiFi processes the same data more than once. This may happen when a flow is restarted, a file is reprocessed, or the source system sends the same record repeatedly.

Another common cause is the absence of unique identifiers or duplicate checking before inserting records into the analytics database. If NiFi does not track already processed files or records, the same data may be inserted multiple times.

To solve this problem, each record should have a unique ID or business key. Checksums or hashes can also be used to identify duplicate files or records. NiFi processors should be configured to track processed data and avoid unnecessary reprocessing.

The analytics database should also use primary keys or unique constraints to prevent duplicate records. For database sources, Change Data Capture (CDC) can be used to capture only new or changed records.

The ETL process should be **idempotent**, meaning that processing the same input more than once should not create additional duplicate records.

Conclusion: Unique identifiers, deduplication, NiFi flow tracking, database constraints, CDC, and idempotent ETL processing can eliminate duplicate records.

4. YARN Resource Starvation with Multiple Users

Resource starvation occurs when multiple users submit jobs and some applications consume most of the available CPU and memory resources. This usually indicates poor scheduling configuration or the absence of resource limits and queue management.

If all jobs are submitted to a single queue with equal or uncontrolled resource access, large jobs can consume most cluster resources. Smaller jobs may remain waiting for a long time.

A better approach is to use the YARN Capacity Scheduler or Fair Scheduler. Separate queues can be created for different users, departments, or workloads. Resource capacity and maximum resource limits can be assigned to each queue.

For example, ETL, reporting, and machine-learning workloads can have separate queues. Important jobs can receive higher priority, while resource limits prevent one user from consuming the entire cluster.

YARN should also monitor CPU and memory usage and enforce container resource limits. User and queue quotas can prevent resource monopolization.

Conclusion: The main problem is poor resource scheduling. Queue-based scheduling, fair resource allocation, priorities, quotas, and container limits can prevent resource starvation and improve cluster utilization.

5. Backup Restores Outdated Data After Recovery

Restoring outdated data indicates a problem with the organization's backup or replication strategy. The backup system may not be running frequently enough, or the latest backup may not have been completed successfully before the failure.

Another possible problem is that the organization depends only **on** replication instead of maintaining independent backups. Replication protects against hardware failure but may also replicate accidentally deleted or corrupted data.

To correct this problem, the organization should implement regular and automated backups with clearly defined backup schedules. Incremental backups can be performed frequently, while full backups can be performed periodically.

Multiple backup versions should be retained so that administrators can restore the most recent valid version. Backup timestamps and integrity checks should be used to verify that backups are complete and usable.

A separate backup location should be maintained so that failure of the main storage system does not destroy both the original data and its backup. Recovery testing should also be performed regularly.

Conclusion: The outdated recovery is caused by an inadequate backup frequency, poor version management, or over-reliance on replication. Automated backups, versioning, independent storage, integrity checks, and regular recovery testing provide reliable and up-to-date data recovery

Error Analysis Task Rubric

		and feasible corrections	corrections		corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented